



10

High-Performance State Updates

State represents the dynamic aspect of your React application. When the state changes, your components react to those changes. Without state, you would have nothing more than a fancy HTML template language. Usually, the time required to perform a state update and have the changes rendered on the screen is barely, if at all, noticeable. However, there are times that complex state changes can lead to noticeable lag for your users. The goal of this chapter is to address these cases and find out how we can avoid those lags.

In this chapter, you'll learn how to do the following:

- Batch your state changes together for minimal re-rendering
- Prioritize state updates to render content that's critical for your user experience first
- Develop strategies for performing asynchronous actions while **batching** and prioritizing **state updates**

Technical requirements

For this chapter, you'll need your code editor (**Visual Studio Code**). The code we'll be following can be found here:

<https://github.com/PacktPublishing/React-and-React-Native-5E/tree/main/Chapter10>.

You can open the terminal within Visual Studio Code and run `npm install` to make sure you're able to follow along with the examples as you read through the chapter.

Batching state updates

In this section, you'll learn about how React can **batch state** updates together to prevent unnecessary rendering when multiple state changes happen simultaneously. In particular, we'll look at the changes introduced in **React 18** that make **automatic batching** of state updates commonplace.

When your React component issues a state change, this causes the React internals to re-render the parts of your component that have changed visually as a result of this state update. For example, imagine you have a component with a name state that's rendered inside of a `<span>` element, and you change the name state from `Adam` to `Ashley`. That's a straightforward change that results in a re-render that's too fast for the user to even notice. Unfortunately, state updates in web applications

are rarely this straightforward. Instead, there might be dozens of state changes in 10 milliseconds. For example, the name state might follow changes like this:

1. Adam
2. Ashley
3. Andrew
4. Ashley
5. Aaron
6. Adam

Here, we have six changes that took place with the name state in a short amount of time. This means that React would have re-rendered the **DOM** six times, once for each value that was set as the name state. What's interesting to note about this scenario is the final state update: we're back where we started with `Adam`. This means that we just re-rendered the DOM five times for no reason. Now, imagine these wasted re-renders on a web application scale and how these types of state updates might cause problems for performance. For example, when the app uses complex animations, user interactions like drag and drops, timeouts, and intervals can all lead to unnecessary re-renders, negatively impacting performance.

The answer to this problem is **batching**. This is how React takes several state updates that were made in our component

code and treats them as a single state update. Rather than process every state update individually, while re-rendering the DOM between each update, the state changes are all merged, which results in one DOM re-render. In the aggregate, this reduces the amount of work that our web applications need to do by a lot.

In **React 17**, automatic batching of state updates only happened inside of event handler functions. For example, let's say you have a button with an `onClick()` handler that performs five state updates. React will batch all of these state updates together so that only one re-render is necessary. The problem arises when your event handlers make asynchronous calls, usually to fetch some data, and then make state updates when the asynchronous call finishes. These state changes are no longer automatically batched because they're not running directly inside of the event handler function. Instead, they're running in the `callback` code of the asynchronous operation and React 17 will not batch these updates. This is a challenge because it's common for our React components to fetch data asynchronously and perform state updates in response to events!

Now we know how we can handle the most common problem with unnecessary re-renders, which is multiple changes to a state in a short time. Now, let's understand it by example.

React 18 batching

Let's turn our attention to some code now to see how **React 18** addresses the batching problem that we've just outlined. For this example, we'll render a button that, when clicked, will perform 100 state updates. We'll use `setTimeout()` so that the updates are performed asynchronously, outside of the event handler function. The idea is to show the difference between how this code is handled by two different React versions. To do this, we can open the **React profiler** in the browser dev tools and hit record before we press the button to execute our state changes. Here's what the code looks like:

```
import * as React from "react";
export default function BatchingUpdates() {
  let [value, setValue] = React.useState("loading...");
  function onStart() {
    setTimeout(() => {
      for (let i = 0; i < 100; i++) {
        setValue('value ${i + 1}');
      }
    }, 1);
  }
  return (
    <div>
      <p>
        Value: <em>{value}</em>
      </p>
    </div>
  );
}
```

```
    <button onClick={onStart}>Start</button>  
  </div>  
);  
}
```

By clicking the button that this component renders, we call the `onStart()` event handler function defined by our component. Then, our handler calls `setValue()` 100 times inside a loop. Ideally, we do not want to perform 100 re-renders because this will hurt the performance of our application, and it doesn't need to. Only the final call to `setValue()` matters here.

Let's first take a look at the profile captured for this component using React 17:

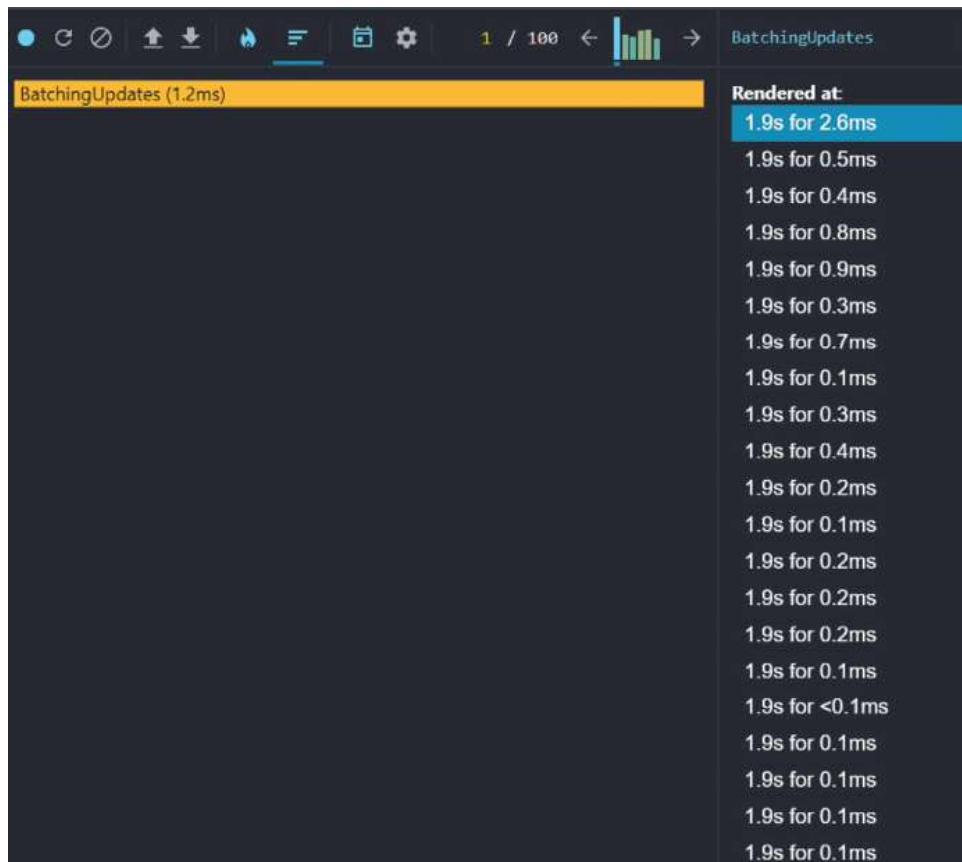


Figure 10.1: Using React dev tools to view re-renders every time state updates are made

By pressing the button with our event handler attached to it, we're making 100 state update calls. Since this is done outside of the event handler function in `setTimeout()`, automatic batching doesn't happen. We can see this in the profile output of the `BactchingUpdates` component, where there's a long list of renders. Most of these aren't necessary and contribute to the

amount of work React needs to do in response to user interactions, hurting the overall performance of our application.

Let's capture a profile of the same component being rendered using React 18:

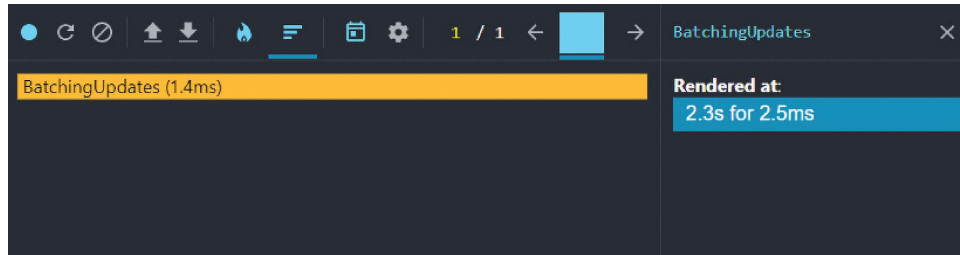


Figure 10.2: React dev tools showing only one render with automatic batching enabled

Automatic batching is applied everywhere that state updates are made, even in common asynchronous scenarios such as this one. As the profile shows, there's only one re-render when we click the button instead of 100. We didn't have to make any adjustments to our component code to make this happen either. However, there is one change that's required in order to make state updates batch automatically. Let's say you used `ReactDOM.render()` to render your root component, like so:

```
ReactDOM.render(  
  <React.StrictMode>
```

```
    <App />
  </React.StrictMode>,
  document.getElementById("root")
);
```

Instead, you can use `ReactDOM.createRoot()` and render that:

```
ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

By creating and rendering your root node this way, you can ensure that with **React 18**, you'll get batched state updates throughout your application. You no longer need to worry about manually optimizing state updates so that they take place immediately: React does this for you now. However, sometimes, you'll have state updates that are of higher priority than others. In cases like these, we need a way to tell React to prioritize certain state updates over others, instead of batching everything together.

Prioritizing state updates

When something happens in our React application, we usually make several state updates so that the UI can reflect these

changes. Typically, you can make these state changes without much thought about how the rendering performance is impacted. For example, let's say you have a long list of items that need to be rendered. This will probably have some impact on the UI: while the list is being rendered, the user probably won't be able to interact with certain page elements because the JavaScript engine is 100% utilized for a brief moment.

However, this can become an issue when expensive rendering disrupts the normal browser behavior that users expect. For example, if the user types in a text box, they expect the character they just typed to show up immediately. But if your component is busy rendering a large item list, the text box state cannot be updated right away. This is where the new React state update prioritization API comes in handy.

The `startTransition()` API is used to mark certain state updates as transitional, meaning that the updates are treated as a lower priority. If you think about a list of items either being rendered for the first time or being changed to another list of items, this is a transition that doesn't have to be immediate. On the other hand, state updates such as changing the value in a text box should be as close to immediate as possible. By using `startTransition()`, you tell React that any state updates within can wait if there are more important updates.

A good rule of thumb for `startTransition()` is to use it for the following:

- Anything that has the potential to perform a lot of rendering work
- Anything that doesn't require immediate feedback from the user in response to their interactions

Let's walk through an example that renders a large list of items in response to a user typing in a text box to filter a list.

This component will render a text box that the user can type in to filter a list of 25,000 items. I've chosen this number based on the performance of the laptop I'm using to write this code: you might want to tweak it up if there's no delay or down if it takes too long to render anything. When the page first loads, you should see a filter text box that looks like this:



Figure 10.3: The filter box before the user types anything

When you start typing in the filter text box, the filtered items will render underneath it. It might take a second or two, since there are so many items to render:

- Item 1
- Item 10
- Item 11
- Item 12
- Item 13
- Item 14
- Item 15

Figure 10.4: Filtered items underneath the filter input when the user starts typing

Now, let's walk through the code, starting with a large array of items:

```
let unfilteredItems = new Array(25000)
  .fill(null)
  .map((_, i) => ({ id: i, name: 'Item ${i}' }));
```

The size of the array is specified in the array constructor, and then it's filled with numbered string values that we can filter by.

Next, let's look at the state used by this component:

```
let [filter, setFilter] = React.useState("");  
let [items, setItems] = React.useState([]);
```

The `filter` state represents the value of the filter text box and defaults to an empty string. The `items` state represents the filtered items from our `unfilteredItems` array. This array is populated when the user types in the filter text box.

Next, let's look at the markup rendered by this component:

```
<div>  
  <div>  
    <input  
      type="text"  
      placeholder="Filter"  
      value={filter}  
      onChange={onChange}  
    />  
  </div>  
  <div>  
    <ul>  
      {items.map((item) => (  
        <li key={item.id}>{item.name}</li>  
      ))}  
    </ul>  
  </div>  
</div>
```

The filter text box is rendered by an `<input>` element, while the filtered results are rendered as a list by iterating over the `items` array.

Finally, let's look at the event handler function that's fired when the user types in the filter text box:

```
const onChange = (e) => {  
  setFilter(e.target.value);  
  setItems(  
    e.target.value === ""  
    ? []  
    : unfilteredItems.filter((item) => item.name.includes(e.target.value))  
  );  
};
```

The `onChange()` function is called when the user types in the filter text box and sets two state values. First, it uses `setFilter()` to set the value of the filter text box. Then, it calls `setItems()` to set the filtered items to render unless the filter text is empty, in which case, we render nothing.

When interacting with this example, you might notice a problem with the responsiveness of the text box when typing in it. This is because, in this function, we're setting not only the text box value but also the filtered items. This means that before

the text value can be rendered, we have to wait for thousands of items to be rendered.

Even though these are two separate state updates (`setFilter()` and `setItems()`), they're batched and treated as a single state update. Likewise, when the rendering starts, React makes all the changes at once, which means that the CPU won't let the user interact with the text box because it's fully utilized, rendering the long list of filter results. Ideally, we want to prioritize the text box state update while letting the items render afterward. To put it another way, we want to deprioritize the item rendering, since it's expensive and the user doesn't interact with it directly.

This is where the `startTransition()` API comes in. Any state updates that take place within the function that's passed to `startTransition()` will be treated with lower priority than any state updates that happen outside of it. In our filtering example, we can fix the text box responsiveness issue by moving the `setItems()` state change inside of `startTransition()`.

Here's what our new `onChange()` event handler looks like:

```
const onChange = (e) => {  
  setFilter(e.target.value);  
  React.startTransition(() => {  
    setItems(  

```

```
    e.target.value === ""
      ? []
      : unfilteredItems.filter((item) => item.name.includes(e.target.value))
    );
  });
};
```

Note that we didn't have to make any changes to how the item's state is updated: the same code is moved to a function that's passed to `startTransition()`. This tells React to only execute this state change after any other state changes are complete. In our case, this allows the text box to update and render before the `setItems()` state change runs. If you run the example now, you'll see that the responsiveness of the text box is no longer affected by how long it takes to render a long list of items.

Before this new API was introduced, you could achieve state update prioritizations via workarounds with `setTimeout()`. The main disadvantage of this approach is that the internal React scheduler knows nothing about your state updates and their priorities. For example, by using `startTransition()`, React can cancel the update entirely if the state changes again before completion or if the component is unmounted.

In real applications, it isn't simply a matter of prioritizing which state updates should run first. Rather, it's a combination

of fetching data asynchronously while making sure that priorities are taken into account. In the final section of this chapter, we'll tie all of this together.

Handling asynchronous state updates

In this final section of the chapter, we'll look at the common scenario of fetching data asynchronously and setting render priorities. The key scenario that we want to address is making sure that users aren't interrupted when typing or doing any other interaction that requires immediate feedback. This requires both proper prioritization and handling asynchronous responses from the server. Let's start by looking at the React APIs that can potentially help with this scenario.

The `startTransition()` API can be used as a **Hook**. When we do this, we also get a Boolean value that we can check to see whether the transition is still pending. This is useful for showing the user that things are loading. Let's modify the example from the previous section to use an asynchronous data-fetching function for our items. We'll also use the `useTransition()` Hook and add loading behavior to the output of our component:

```
let unfilteredItems = new Array(25000)
  .fill(null)
  .map((_, i) => ({ id: i, name: 'Item ${i}' }));
function filterItems(filter: string) {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve(unfilteredItems.filter((item) => item.name.includes(filter)));
    }, 1000);
  });
}
export default function AsyncUpdates() {
  const [isPending, startTransition] = React.useTransition();
  const [isLoading, setIsLoading] = React.useState(false);
  const [filter, setFilter] = React.useState("");
  const [items, setItems] = React.useState<{ id: number; name: string }[]>([]);
  const onChange: React.ChangeEventHandler<HTMLInputElement> = async (e) => {
    setFilter(e.target.value);
    startTransition(() => {
      if (e.target.value === "") {
        setItems([]);
      } else {
        filterItems(e.target.value).then((result) => {
          setItems(result);
        });
      }
    });
  };
  return (...);
}
```

What this example shows is that once you start typing in the filter text box, it will trigger the `onChange()` handler, which will call the `filterItems()` function. We also have an `isLoading` value that we can use to show the user that something is happening in the background:

```
<div>
  <div>
    <input
      type="text"
      placeholder="Filter"
      value={filter}
      onChange={onChange}
    />
  </div>
  <div>
    {isPending && <em>loading...</em>}
    <ul>
      {items.map((item) => (
        <li key={item.id}>{item.name}</li>
      ))}
    </ul>
  </div>
</div>
```

Here's what the user will see when `isLoading` is `true`:



Figure 10.5: A loading indicator while a state transition is pending

However, there's a slight problem with our approach. You might have noticed the loading message flash briefly when typing into the text box. But then, you probably had a longer period when the items still weren't visible, and the loading message disappeared. What's happening here? Well, the `isPending` value that comes from the `useTransition()` Hook can be misleading. We've designed our component in such a way that `isPending` will be `true` in the following situations:

- If the `filterItems()` function is still fetching our data
- If the `setItems()` state update is still performing an expensive render with lots of items

Unfortunately, this isn't how `isPending` works. The only time this value is `true` is before the function we pass to `startTransition()` is run. This is why you'll see the loading indicator flash briefly instead of being displayed throughout the data-fetching operation and the rendering operation.

Remember, React schedules state updates internally, and by using `startTransition()`, we've scheduled `setItems()` to run after other state updates.

Another way to think about `isPending` is that it's `true` while high-priority updates are still running. We can call it `highPriorityUpdatesPending` to avoid confusion. That said, the uses of this value are narrow, but they do happen from time to time. For our more common case of fetching data and performing an expensive render, we need to think of another solution. Let's walk through our code and refactor it in such a way that the loading indicator is displayed while the fetch and the higher-priority updates happen. First, let's introduce a new `isLoading` state that defaults to `false`:

```
const [isLoading, setIsLoading] = React.useState(false);  
const [filter, setFilter] = React.useState("");  
const [items, setItems] = React.useState([]);
```

Now, inside of our `onChange()` handler, we can set the state to `true`. Inside of the transition that runs after the data fetch completes, we set it back to `false`:

```
const onChange: React.ChangeEventHandler<HTMLInputElement> = async (e) => {  
  setFilter(e.target.value);  
  setItems(fetchItems(filter));  
  setPending(true);  
  startTransition(() => {  
    setItems(fetchItems(filter));  
  });  
  setPending(false);  
};
```

```
React.startTransition(() => {
  if (e.target.value === "") {
    setItems([]);
    setIsLoading(false);
  } else {
    filterItems(e.target.value).then((result) => {
      setItems(result);
      setIsLoading(false);
    });
  }
});
};
```

Now that we're keeping track of the `isLoading` state, we know exactly when all the heavy lifting is done and can hide the loading indicator. The final change is to base the indicator display on `isLoading` instead of `isPending`:

```
<div>
  {isLoading && <em>loading...</em>}
  <ul>
    {items.map((item) => (
      <li key={item.id}>{item.name}</li>
    ))}
  </ul>
</div>
```

When you run the example with these changes, the results should be a lot more predictable. The `setLoading()` and `setFilter()` state updates are high-priority and execute immediately. The call to fetch data using `filterItems()` isn't made until the high-priority state updates are completed.

Only after we have the data do we hide the loading indicator.

Summary

This chapter introduced you to the new APIs available in React 18 that help you achieve high-performance state updates. We started with a look at the changes to automatic state update batching in React 18 and how to best take advantage of them. We then explored the new `startTransition()` API and how it can be used to mark certain state updates as having a lower priority than those that require immediate feedback for user interactions. Finally, we looked at how state update prioritization can be combined with asynchronous data fetching.

In the next chapter, we'll go over fetching data from the server.